

Lecture 2

Recap:

Recap: Think/Remember an example case: Why is testing necessary?: The Ariane 5 crisis where a failure in the software happening by numerical conversion error lead to the rocket exploding only 39 seconds after it was launched that costed the company millions and hurt its reputation.

Recap: Think/Remember an example case: Why is automated testing necessary?: automated tests which is part of the analytical quality assurance helps the team catch errors before they are sent to the user. It helps avoid dangerous mistakes.

N.B: you can never test every case for the software, saying “we have tested everything, and every case is misleading and very dangerous”

How do others test? (Automotive):

Component Test (Incoming Test of Components)

- **What it is:** Test individual parts as they arrive from suppliers (e.g., sensors, ECUs).
- **Purpose:** Verify that each component works correctly before assembly.
- **Software analogy:** Unit testing — testing one module or function.

Integration Test (Ongoing Intermediate Checks on the Assembly Line)

- **What it is:** Check how different components work together while assembling the car.
- **Purpose:** Detect problems caused by interactions between parts.
- **Software analogy:** Integration testing — checking combined modules for errors.

System Test (Test in Use Under (Almost) Real-World Conditions)

- **What it is:** Test the whole car in controlled conditions (test tracks, labs).
- **Purpose:** Ensure that the complete system works as intended under realistic conditions.

- **Software analogy:** System testing — end-to-end tests of the entire application.

Acceptance Test (Customer Test Drive)

- **What it is:** Let real customers try the car before production approval.
- **Purpose:** Verify that the car meets customer expectations and requirements.
- **Software analogy:** User acceptance testing (UAT).

Performance Test / Load Test (Race Use)

- **What it is:** Test car under extreme use (high speeds, heavy loads, racing).
- **Purpose:** Measure stability, reliability, and robustness.
- **Software analogy:** Performance, load, and stress testing.

Stress Test (Crash Test)

- **What it is:** Test how the car behaves under extreme conditions or failures (crashes).
- **Purpose:** Ensure safety and system robustness even under catastrophic events.
- **Software analogy:** Stress testing — see how software behaves under extreme load or failure.

Is a complete test possible:

A program that takes 3 numbers as an input and perform the same task on them, the input type is of 16-bits that means for each input we have 2^{16} possibilities meaning we have $2^{16} * 2^{16} * 2^{16}$ which is total of 2^{58} making it total of 281.474.976.710.656 possibilities. If we assumed that we can do 100,000 test cases per second we need 90 years to finish it.

We have a program which is a loop that has 5 if statements inside, the loop can run any amount of times up to 20. Which means we have 5 choices and up to 20 times. This makes it calculate by $5^1 + 5^2 + \dots + 5^{18} + 5^{19} + 5^{20} = 119.209.289.550.780$ test cases Duration: about 38 years.

We have a third program where we take 3 positive integer inputs, and the program decides if the triangle is equilateral, isosceles, or unequal. As proven before test case are impossible to do all it takes years.

Effective Testing:

While testing its impossible to come up with all the test cases, so you should focus on making test cases that might blow your program.

1. Permutations:

- Test all input orders to ensure classification works regardless of input sequence.

2. Invalid Inputs:

- Include zeros, negative numbers, non-integers, letters, or special characters to check error handling.

3. Boundary Cases:

- Test extreme and edge conditions, e.g., maximum integers, sum of two sides equals the third, all sides zero.

4. Coverage:

- Use representative test cases covering equivalence classes and boundaries; exhaustive testing is impossible.

5. Professional Testing Approach:

- Select a small set of effective test cases to catch most defects efficiently rather than attempting infinite combinations.

N.B: the program (problem) many times seem easy to solve but the testing is much harder.

The Test:

Determines speed of development

- You cannot deploy untested software, the late you discover the issues the late the project is finished. Testing sets the rhythm of the development.

Gets more complex with program complexity

- As software grows, the number of possible inputs, paths, and interactions grows exponentially and so is the complexity of the testing.

Gets more important with relevance

- Takeaway: The more critical the software, the more thorough the testing must be.

Is far from complete

- Smart selection of tests is essential; you cannot test everything.

Is much more than “clicking around”

- Random testing or casual usage is not enough.
- Professional testing is systematic, planned, and goal-oriented.

Needs experts

- Effective testing requires knowledge, experience, and sometimes certifications (e.g., CTFL).

The Chain of Causes and Errors

an error is a mistake that has been made by the developer (either logical, or compile time issue) the defect is that the software will give a wrong output or crash, this crash then is called failure or the wrong output.

Error exists in the developer's mind/code.

Defect exists in the software product, before it even causes a visible problem (failure).

Root cause: the misunderstanding that lead to this failure

a programmer creates an error when the error is written down as a code its called a defect when the code is ran and give bad output its called failure (the output).

if the output is wrong but i still dont know what the correct output should be, this is not called error yet

Failure:

N.B: sometimes failures happen because of hardware issues, or waves and rays. And it can lead to abortion of the task or a completely wrong output.

A failure happens when the system fails to fulfil a task within the defined scope during its execution. compared to what were the requirements that we have had specified.

Example: if we stated in the requirements the system should handle 10MB file, if the user uploads a 50MB file and the system fails rejects it, its not a failure since this was not a requirement. But if the system crashed then yes it's a failure.

A failure is what the user, tester are able to see from the outside interaction with the system.

It can be also called a deviation from what the system is expected to do.

Defect:

A defect is an inadequacy in the work product (documentation, code, product design) that will make it not fulfill requirements. Something is wrong, missing, incomplete, or incorrect. A defect only exist because there are requirements to proof its wrong.

Defects happen cause of 3 reasons:

Problem with data definition: the developer chose the wrong data type, or allowed it to happen.

Incorrect instruction: certain part of the code is logically wrong.

Incorrect software module as a whole: a certain module or class behaves wrongly.

A defect can be existent but only noticeable under certain system pressure. In other words. The program behaves normally and correctly and crashes under pressure. To give an example: the software works perfectly with 100 users using it and once 200 users start using it, it crashes.

Error:

An error is a human action or omission that causes an incorrect result, potentially leading to a defect or failure.

The human who does the error can be both the user or the developer.

Developer: lack of validation, logical error.

User: invalid input, forgets to do some operation.

Error happens when there is something omitted (not done) or something is done but in a wrong way. Example a developer forget to validate the data (omission) the developer writes the wrong logic. This can happen by the person accidentally on purpose or without knowing.

Catching: Error, Defect, Failure:

Error: we can detect and prevent it by pushing standards and training to our developers and users.

Standards: naming conventions, documentation standards.

Training: Teaching developers and users how to their tasks correctly.

Defect: We can catch and prevent it by static testing and analysis, and by debugging.

Debugging: Debugging is an activity of locating/identifying, analyzing and removing the causes of failures in the software.

Failure: We can detect it by dynamic testing and analysis.

Testing:

Testing is the process of planning, preparing, and evaluating a component or system (and related work products) to check whether it meets requirements, is fit for purpose, and is free of errors. Including all static and dynamic processes.

Planning

Decide what to test, how, and with what resources

Example: Plan test cases for a login feature, including valid and invalid credentials

Preparation

Set up the test environment, data, and tools

Example: Prepare a test database with users, set up automated test scripts

Evaluation

Execute tests and check results against expected outcomes

Enter valid credentials, expect login success; enter invalid, expect error message.

Process goal: to see if it works as it is intended, if they meet the specific requirements, if they are error free.

Test Objectives:

Uncovering defects and faults: find mistakes in the code, the design, or the architecture like finding a bug in the code that calculate the interest rate in an app.

Evaluate work results: requirements, user stories, architecture design and code: check all the work artifacts including documentation for errors. Check the architecture of a mobile app to make sure that it works offline.

Identify errors in the work results and avoid them in subsequent work results: learn from detected errors to prevent them in the future. A developer forgets to add validation for the input, so the team documents it and add it to the standards for later use.

Verify whether all specific requirements have been met Check completeness of the test object: ensure each requirement was implemented and has test cases. Verify that password reset functionality works for both email and SMS.

Validate the test object against user and other stakeholder expectations: ensure that the software does what the stakeholders and users expect it to be.

Create confidence in the quality level of the test object: Demonstrate software is reliable, safe, and fit for purpose. This can be done through stress tests.

Reduce risks due to inadequate software quality: Identify and mitigate risks before software is released. Load testing.

Provide stakeholders with information to make informed decisions, especially regarding the quality level of the test object: Report test results so stakeholders can decide on release or corrective actions.

Verification:

Testing if we are building the app as we have agreed on, if we are following our requirements.

Confirmation by providing objective evidence that specified requirements have been met.

Testing against requirements, are we doing it right.

Validation:

Actually testing the app after its built to see if it works correctly. Test against application context, are we doing the right thing.

Confirmation by providing objective evidence that the requirements for a specific intended use or application have been met.

What is Quality?:

Quality is about meeting both explicit requirements and expectations of all stakeholders. So the more requirements and expectations it meets the higher the quality.

“Degree to which a set of inherent characteristics fulfills requirements” (ISO 9000:2000):

Inherent characteristics: Built-in properties of the system, such as functionality, performance, reliability, usability, and security.

Fulfills requirements: The system satisfies the needs or specifications defined for it.

Degree: Quality is measured by how well these characteristics meet the requirements.

This includes the functional and non-functional requirements.

Functional: Actions, tasks, calculations, or processes the system must perform.

Non-functional: System performance, reliability, usability, security, maintainability.



System reliability:

Reliability is about doing what it should, even when conditions are tough. Stress testing is one way to verify reliability under extreme condition

Reliability is the ability of a system to perform its intended functions correctly over time, without failing, under specified conditions.